

PROVA FINAL

- Entrega Sem prazo
- Pontos 40
- Perguntas 10
- Limite de tempo 60 Minutos
- Tentativas permitidas 2

Instruções

INSTRUÇÕES DA PROVA FINAL

- A prova tem a duração de 60 minutos.
- Ao clicar em PROVA FINAL, no menu “Testes” você iniciará a prova. Ao acessar a página com as questões, o tempo começa a ser contado.
- A prova é composta de 10 (dez) questões objetivas, sendo 04 (quatro) pontos cada.
- Ao final do teste não se esqueça de enviá-lo, clicando no botão “ENVIAR TESTE”. Só utilize esse botão quando tiver finalizado a avaliação.
- Se necessário, durante a prova, entre em contato pelo link “Atendimento”.
- Você terá 02 (duas) tentativas e, caso necessite de uma nova, será preciso solicitar a prova extra, que requer pagamento de taxa adicional.

Atenção: Todas as provas iniciadas e que não houverem sido submetidas serão automaticamente encerradas pelo sistema transcorridos os 60 minutos de duração.

Boa prova!

Histórico de tentativas

	Tentativa	Tempo	Pontuação
MANTIDO	Tentativa 2	3 minutos	40 de 40
MAIS RECENTE	Tentativa 2	3 minutos	40 de 40
	Tentativa 1	8 minutos	36 de 40

Pontuação desta tentativa: 40 de 40

Enviado 7 jan em 19:19

Esta tentativa levou 3 minutos.

Resposta correta



Pergunta 1

4 / 4 pts

A separação de responsabilidades proposta pelo padrão **CQRS** (Command and Query Responsibility Segregation) torna-se especialmente útil em aplicações que enfrentam problemas de escalabilidade e performance. Separando os modelos de leitura e escrita, é possível usar tecnologias diferentes, aplicar otimizações específicas e escalar operações de forma independente. No entanto, isso também aumenta a complexidade da aplicação, exigindo maior cuidado na fase de planejamento.

Qual benefício justifica a adoção da arquitetura CQRS em um sistema de médio a grande porte?

- ☐ Redução da duplicação de código e simplificação do modelo de dados, ainda que com performance reduzida.
- ☐ Adoção de apenas um tipo de banco de dados, garantindo padronização na infraestrutura.
- ☒ Otimização de performance por permitir que comandos (escrita) e queries (leitura) tenham modelos separados, ajustados às suas necessidades, com possibilidade de escalabilidade independente.
- ☐ Centralização das operações de leitura e escrita para garantir consistência em tempo real.
- ☐ Substituição da arquitetura REST por sockets assíncronos em todas as operações.

Resposta correta



Pergunta 2

4 / 4 pts

Ao implementar cache com Redis em uma aplicação NestJS, é possível desbloquear algumas funcionalidades em produção, ao passo que também adiciona um pouco mais de complexidade na arquitetura do sistema como um todo.

Assinale a alternativa que melhor descreve os benefícios da utilização de um sistema de cache externo, como por exemplo o Redis:

- ☐ Permite a modelagem de dados através do Typescript
- ☒ Compartilha o cache entre diferentes instâncias, garantindo consistência em ambientes que precisam de ser escaláveis.
- ☐ Obriga o uso de filas de mensageria para sincronização dos dados.
- ☐ Elimina a necessidade de banco de dados, pois os dados uma vez armazenados em cache serão persistidos da mesma forma.
- ☐ O redis salva o cache no frontend para repostas mais rápidas à requisições

Resposta correta



Pergunta 3

4 / 4 pts

Aplicações NestJS podem aplicar cache em endpoints para reduzir o tempo de resposta em chamadas repetidas. Sendo assim, qual o comportamento esperado quando aplicamos

`@CacheTTL(60000)` a um método controller no NestJS?

- ☐ O controller terá uma nova instância iniciada após 1 minuto.

☐ O método não será executado por 60 segundos.



A resposta será armazenada em cache por 60 segundos, abstraindo-se o mecanismo sendo utilizado para controle de cache.

☐ A resposta será armazenada em cache por 60 segundos, caso a aplicação esteja utilizando o Redis.



Os dados não são enviados completos para o frontend, uma vez que a estratégia de cache consiste na divisão dos dados entre front-end e back-end

Resposta correta



Pergunta 4

4 / 4 pts

O sistema de módulos no Node.js permite dividir a aplicação em partes reutilizáveis, facilitando a manutenção e escalabilidade da aplicação. Com base no sistema de módulos, assinale a alternativa **VERDADEIRA**.

☐ Apenas funções podem ser exportadas como módulos.

☐ O sistema de módulos impede o uso de classes.

☒ Qualquer valor (objeto, classe, função, etc.) pode ser encapsulado como módulo.

☐ Um módulo só pode ser importado uma única vez.

☐ É necessário compilar os módulos manualmente.

Resposta correta



Pergunta 5

4 / 4 pts

A Clean Architecture propõe que as regras de negócio fiquem totalmente desacopladas de frameworks, UIs e fontes de dados, o que favorece testabilidade e manutenção. Este tipo de arquitetura possui práticas recomendadas que garantem que a aplicação continue escalável e robusta no decorrer do tempo e de acordo com que novas funcionalidades vão sendo adicionadas.

Assinale a alternativa que melhor descreve os principais pilares da arquitetura CLEAN:

☒ Separação de responsabilidades em camadas, com dependência unidirecional do externo para o interno.



Encapsulamento das requisições websocket, uma vez que o modelo de APIs REST é pouco recomendado em arquiteturas CLEAN



Prevenção à reutilização de código dentro de uma mesma camada, fazendo com que a duplicidade favoreça a robustez da aplicação

☐ Transição de dados e chamadas livre entre as camadas, não importando a sua ordem.

☐ Redução de módulos e maior acoplamento com o banco de dados

Resposta correta



Pergunta 6

4 / 4 pts

O ORM é uma técnica que pode ser utilizada em aplicações de qualquer porte. Essa técnica possibilita que a aplicação evolua de forma mais saudável e adiciona uma série de facilidades na utilização dos métodos pela camada de repositório.

Assinale a alternativa CORRETA a respeito dos ORMs:

☒

O ORM é uma técnica que possibilita a abstração da implementação de acesso a banco. Com isso, os métodos de acesso encapsulam as queries e fazem com que alterações no acesso à dados não afetem a regra de negócios da aplicação.

☐

O ORM é uma técnica que possibilita a abstração da implementação dos controladores. Com isso, os métodos de endpoint encapsulam os caso de uso e fazem com que alterações na regra de negócio não afetem a camada de acesso a dados.

☐

O ORM traz uma complexidade muito grande para as aplicações, não sendo nunca recomendado para projetos de pequeno ou médio porte.

☐

O ORM é uma técnica que possibilita a abstração da implementação dos casos de uso. Com isso, os métodos de regras de negócio encapsulam as chamadas à APIs externas e fazem com que alterações no acesso à dados não afetem a regra de negócios da aplicação.

☐

A técnica do ORM consiste em abstrair a instalação de dependências em uma única biblioteca. Sendo assim, caso haja uma alteração em qualquer dependência externa da aplicação, podemos fazer uma única alteração no package.json, uma vez que as instalações estão abstratidas.

Resposta correta



Pergunta 7

4 / 4 pts

O Node.js utiliza a V8 Engine para executar JavaScript fora do navegador, sendo altamente eficiente em termos de desempenho e consumo de recursos. Qual característica técnica contribui para que o Node.js seja leve e eficiente?

☐

Compilação direta em C.

☐

Execução síncrona com alta prioridade.

☐

Execução de scripts via browser somente.

☒

Arquitetura single-thread baseada em eventos.

☐ Uso exclusivo de multithread.

Resposta correta



Pergunta 8

4 / 4 pts

A seguir, temos a implementação de uma entidade `Invoice` e seu repositório personalizado com métodos específicos para manipulação de faturas:

```
// invoice.entity.ts

@Entity()
export class Invoice {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  clientId: number;

  @Column({ default: false })
  paid: boolean;

  @Column({ type: 'timestamp', nullable: true })
  paidAt?: Date;
}

// invoice.repository.ts

@Injectable()
export class InvoiceRepository {
  constructor(
    @InjectRepository(Invoice)
    private readonly repo: Repository<Invoice>,
  ) {}

  async findUnpaidByClient(clientId: number): Promise<Invoice[]> {
    return this.repo.find({ where: { clientId, paid: false } });
  }

  async markAsPaid(id: number): Promise<Invoice> {
    const invoice = await this.repo.findOne({ where: { id } });
    if (!invoice) throw new NotFoundException();
    invoice.paid = true;
    invoice.paidAt = new Date();
    return this.repo.save(invoice);
  }
}
```

Em relação à implementação apresentada, qual alternativa expressa corretamente boas práticas e uso adequado do padrão Repository no NestJS?

☐ O uso de `@InjectRepository()` é inadequado para entidades com múltiplos campos.

☐

O repositório está assumindo funções indevidas do service, pois trata estados e exceções que não pertencem à camada de persistência.

☒

A abordagem respeita a separação de responsabilidades, pois o repositório encapsula o acesso ao banco e expõe operações específicas sem violar regras de domínio.

☐

A entidade deveria conter lógica interna de validação, como lançar exceções diretamente nos métodos `setPaid()`.

☐ A lógica de marcação de faturas como pagas deveria estar fora do repositório e delegada ao controller

Resposta correta

⋮

Pergunta 9

4 / 4 pts

O padrão repositório é amplamente usado para separar a lógica de acesso ao banco da lógica de negócio. Isso tem uma série de impactos na aplicação no longo prazo. Sendo assim, assinale a alternativa correta sobre a manutenção de aplicações que utilizam o padrão repositório:

☐ Garante que apenas queries SQL sejam usadas.

☒ Facilita a substituição do banco de dados sem grandes alterações na lógica da aplicação.

☐ Permite que o frontend acesse diretamente os dados.

☐ Substitui a necessidade de validações no controller.

☐ Elimina a necessidade de DTOs e serviços.

Resposta correta

⋮

Pergunta 10

4 / 4 pts

A autenticação baseada em JWT (JSON Web Token) é amplamente adotada em APIs modernas. Ao contrário da abordagem de sessões armazenadas no servidor, o JWT gera um token assinado e autocontido, que é armazenado no cliente (geralmente em `localStorage` ou cookies) e enviado em todas as requisições subsequentes via cabeçalho HTTP. Isso reduz a necessidade de controle de sessão no backend e favorece arquiteturas stateless e escaláveis. No entanto, a escolha do armazenamento e a forma de uso do token devem ser feitas com atenção para evitar vulnerabilidades, como XSS e CSRF.

Considerando os princípios da autenticação JWT, qual é uma das vantagens centrais desse modelo em relação ao uso tradicional de sessões no servidor?

☐ Permite que o servidor mantenha um cache local de tokens ativos para verificação rápida de login.

- ☐ Permite revogar tokens individualmente por meio de uma denylist no cliente.
- ☐ Garante criptografia ponta a ponta com autenticação de dois fatores.
- ☐ Elimina a necessidade de qualquer forma de validação de usuário no backend.
- ☒

Exige menor controle de estado no backend, pois o token enviado pelo cliente já contém as informações necessárias para autenticação, permitindo aplicações mais escaláveis

Pontuação do teste: 40 de 40